SOKOINE UNIVERSITY

SUA

OF AGRICULTURE

ABE 715: Computer Programming

Compendium Part 3

Data Analysis, Visualization, and Numerical Methods

Prepared for:
MSc. Irrigation Engineering and Management
MSc. Agricultural and Biosystems Engineering

Instructor: Dr. Kadeghe G. Fue
Electronics and Precision Agriculture Lab

Department of Agricultural Engineering
Sokoine University of Agriculture (SUA)

Academic Year 2025/2026

Contents

Preface ii

1 Data Analysis: NumPy & Pandas 1
- 1.1 The Limitation of Standard Python . . . 1
- 1.2 NumPy: Numerical Python . . . 1
  - 1.2.1 Basic Array Operations . . . 1
- 1.3 Pandas: Python Data Analysis . . . 2
  - 1.3.1 Loading Data . . . 2
  - 1.3.2 Data Inspection and Statistics . . . 2
  - 1.3.3 Filtering and Querying . . . 2
- 1.4 Time Series Analysis . . . 3
- 1.5 Chapter 7 Review Questions . . . 3

2 Data Visualization 4
- 2.1 The Importance of Visualization . . . 4
- 2.2 Matplotlib . . . 4
  - 2.2.1 Basic Line Plot . . . 4
  - 2.2.2 Subplots . . . 5
- 2.3 Seaborn . . . 5
  - 2.3.1 Heatmaps . . . 5
- 2.4 Chapter 8 Review Questions . . . 6

3 Numerical Methods I: Root Finding 7
- 3.1 The Analytical Barrier . . . 7
- 3.2 The Bisection Method . . . 7
  - 3.2.1 Algorithm . . . 7
  - 3.2.2 Trace Table Example . . . 8
- 3.3 The Newton-Raphson Method . . . 8
  - 3.3.1 Formula . . . 8
  - 3.3.2 Python Implementation . . . 8
- 3.4 Using SciPy Optimize . . . 8
- 3.5 Chapter 9 Review Questions . . . 9

Comprehensive Assignment 3 10

Preface to Part 3

In Part 3, we transition from "Programming Logic" to "Engineering Analysis."

Modern agricultural engineering is data-driven. We no longer rely on singular manual measurements; we deal with streams of data from weather stations, soil sensors, and satellite imagery. To handle this, standard Python lists are insufficient. We introduce the **Data Science Stack**: **NumPy** for mathematical speed and **Pandas** for data manipulation.

Once we have analyzed data, we must communicate it. We will explore **Matplotlib** and **Seaborn** to generate professional engineering plots suitable for reports and publications.

Finally, we begin our journey into **Numerical Methods**. Many engineering equations (like Manning's Equation for channel depth) cannot be solved analytically. We will learn how to write algorithms that "guess" the answer with high precision using **Root Finding** techniques.

Learning Objectives for Part 3:

- Utilize NumPy and Pandas to clean and analyze large engineering datasets.
- Create professional visualizations to interpret trends and correlations.
- Understand the mathematical derivation of Bisection and Newton-Raphson methods.
- Implement root-finding algorithms to solve implicit engineering equations.

Chapter 1

Data Analysis: NumPy & Pandas

1.1 The Limitation of Standard Python

In Parts 1 and 2, we used Python lists (e.g., `[1, 2, 3]`). While flexible, lists are slow for mathematical operations. If you have a list of 1 million rainfall readings and want to convert them from inches to millimeters, a standard `for` loop is computationally expensive because Python checks the data type of every single number in the list during the loop.

1.2 NumPy: Numerical Python

NumPy introduces the **Array**. Unlike lists, Arrays are:

- **Homogeneous:** All items must be the same type (usually float or int).
- **Contiguous:** Stored in a single block of memory.
- **Vectorized:** Math operations are applied to the whole block at once (SIMD), making it 50-100x faster than loops.

1.2.1 Basic Array Operations

```
import numpy as np

# Standard List (Slow way)
rainfall_in = [1.2, 0.5, 3.1]
rainfall_mm = []
for r in rainfall_in:
    rainfall_mm.append(r * 25.4)

# NumPy Array (Fast way)
arr_in = np.array([1.2, 0.5, 3.1])
arr_mm = arr_in * 25.4  # No loop needed!
```

Listing 1.1: List vs NumPy Array

1.3 Pandas: Python Data Analysis

While NumPy handles raw numbers, **Pandas** handles structured data (tables). It is built on top of NumPy but adds labels, column names, and indices. It effectively brings Excel-like functionality into code.

The core structure is the **DataFrame** (DF).

1.3.1 Loading Data

Engineers usually work with CSV (Comma Separated Values) or Excel files exported from data loggers.

```
import pandas as pd

# Load a weather station log
df = pd.read_csv("morogoro_weather_2023.csv")

# View first 5 rows
print(df.head())
```

1.3.2 Data Inspection and Statistics

Pandas can instantly summarize massive datasets.

Command	Description
`df.info()`	Shows column types and missing values.
`df.describe()`	Summary stats (Mean, Max, Min, Std Dev).
`df.shape`	Returns (Number of Rows, Number of Cols).
`df.columns`	List of column names.

Table 1.1: Essential Pandas Commands

1.3.3 Filtering and Querying

We often need to isolate specific conditions, such as "Days where rainfall > 10mm".

```
# Syntax: df[ Condition ]

# Filter for rainy days
rainy_days = df[ df["Rainfall_mm"] > 0 ]

# Filter for Hot AND Dry days
stress_days = df[ (df["Temp"] > 30) & (df["Humidity"] < 40) ]

print(f"Total Rainy Days: {len(rainy_days)}")
```

Listing 1.2: Filtering Data

1.4 Time Series Analysis

Agricultural data is almost always tied to time (dates). Pandas has powerful time-series tools.

```
# Convert 'Date' column to datetime objects
df["Date"] = pd.to_datetime(df["Date"])

# Set Date as the Index
df.set_index("Date", inplace=True)

# Resample: Calculate Monthly Averages
monthly_avg = df.resample("M").mean()

# Calculate Annual Total Rainfall
annual_rain = df["Rainfall_mm"].resample("Y").sum()
```

Listing 1.3: Resampling Time Data

1.5 Chapter 7 Review Questions

1. Explain "Vectorization" in the context of NumPy. Why is it faster?
2. If you load a CSV and the numeric columns look like text (Object type), what might be the problem?
3. Write the Pandas command to filter a DataFrame `crop_data` to show only rows where `Yield` is greater than 5.0.

Chapter 2

Data Visualization

2.1 The Importance of Visualization

"A picture is worth a thousand numbers." In engineering, visual inspection is critical for:

- **Quality Control:** Spotting outliers or sensor errors (e.g., a temperature spike to 1000°C).
- **Trend Analysis:** Seeing crop growth curves or water table depletion.
- **Communication:** Presenting findings to clients or policymakers.

2.2 Matplotlib

Matplotlib is the foundational plotting library in Python. It mimics the plotting style of MATLAB.

2.2.1 Basic Line Plot

Used for time series or continuous functions.

```
import matplotlib.pyplot as plt

days = [1, 2, 3, 4, 5]
eto = [4.2, 4.5, 3.8, 5.1, 4.9]

plt.figure(figsize=(10, 6))
plt.plot(days, eto, color='blue', linestyle='--', marker='o')

# Essential Labels
plt.title("Daily Reference Evapotranspiration (ETo)")
plt.xlabel("Day of Month")
plt.ylabel("ETo (mm)")
plt.grid(True)

plt.show() # Display the plot
```

Listing 2.1: Plotting Evapotranspiration

2.2.2 Subplots

Engineers often need to compare variables side-by-side (e.g., Rainfall vs. Soil Moisture).

```
fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)

# Top Plot: Rain
ax1.bar(dates, rainfall, color='blue')
ax1.set_ylabel("Rain (mm)")
ax1.set_title("Weather Station Data")

# Bottom Plot: Moisture
ax2.plot(dates, moisture, color='brown')
ax2.set_ylabel("Soil Moisture (%)")
ax2.set_xlabel("Date")

plt.tight_layout()
plt.savefig("field_report.png", dpi=300) # Save high-res
   image
```

Listing 2.2: Creating Subplots

2.3 Seaborn

Seaborn is built on top of Matplotlib. It provides statistical plots that are aesthetically pleasing and easier to code for complex datasets.

2.3.1 Heatmaps

Heatmaps are excellent for visualizing spatial data, such as a grid of soil samples from a field.

```
import seaborn as sns
import numpy as np

# Simulate a 10x10 grid of soil Nitrogen levels
field_grid = np.random.rand(10, 10)

sns.heatmap(field_grid, cmap="YlGn") # Yellow-Green Colormap
plt.title("Nitrogen Distribution Map")
plt.show()
```

2.4 Chapter 8 Review Questions

1. What is the difference between `plt.plot()` and `plt.bar()`?
2. Why is `dpi=300` used when saving figures?
3. Describe a scenario where a Heatmap is more useful than a Line graph.

Chapter 3

Numerical Methods I: Root Finding

3.1 The Analytical Barrier

In algebra class, we solve equations like $2x - 4 = 0$ easily: $x = 2$. This is an **Analytical Solution**. However, engineering equations are often **Transcendental** or implicit, meaning x cannot be isolated.

Example: Manning's Equation for Channel Depth We calculate Flow Rate (Q) knowing Depth (y) easily:

$$Q = \frac{1}{n} A(y) R(y)^{2/3} \sqrt{S}$$

But if we know Q (design flow) and need to find the required Depth (y), we must solve:

$$f(y) = \frac{1}{n} A(y) R(y)^{2/3} \sqrt{S} - Q = 0$$

The variable y is trapped inside the Area (A) and Hydraulic Radius (R) terms with different powers. We cannot isolate y. We need **Numerical Methods**.

3.2 The Bisection Method

This is a "Brute Force" method based on the Intermediate Value Theorem. If a continuous function changes sign between point a and point b, there must be a root (zero) somewhere between them.

3.2.1 Algorithm

1. Guess two points, a (low) and b (high), such that $f(a) \times f(b) < 0$ (Signs are opposite).
2. Calculate the midpoint: $c = \frac{a+b}{2}$.
3. Evaluate $f(c)$.
4. If $f(c)$ is close to 0, stop.
5. If $f(c)$ has the same sign as $f(a)$, replace a with c. Otherwise, replace b with c.
6. Repeat until the interval is tiny.

3.2.2 Trace Table Example

Solving $x^2 - 4 = 0$. True root is 2. Let $a = 0, b = 5$.

Iter	a	b	c (mid)	f(a)	f(c)	Action
1	0	5	2.5	-4	+2.25	Sign Change at a-c. b=2.5
2	0	2.5	1.25	-4	-2.43	Sign Match at a-c. a=1.25
3	1.25	2.5	1.875	-2.43	-0.48	Sign Match at a-c. a=1.875

Table 3.1: Bisection Trace Table

3.3 The Newton-Raphson Method

The Bisection method is reliable but slow. The **Newton-Raphson** method uses the slope (derivative) of the curve to slide down toward the root much faster.

3.3.1 Formula

$$x_{new} = x_{old} - \frac{f(x_{old})}{f'(x_{old})}$$

Where $f'(x)$ is the derivative of the function.

3.3.2 Python Implementation

```
def newton_raphson(func, deriv, x0, tol=1e-6, max_iter=100):
    x = x0
    for i in range(max_iter):
        fx = func(x)
        if abs(fx) < tol:
            return x # Found root

        dfx = deriv(x)
        if dfx == 0:
            print("Derivative zero error!")
            return None

        x = x - (fx / dfx)

    print("Did not converge")
    return None
```

Listing 3.1: Newton-Raphson Function

3.4 Using SciPy Optimize

In professional practice, we use the optimized libraries in `scipy`.

```python
from scipy.optimize import fsolve

def manning_error(y):
    # Knowns: Q=10, n=0.015, b=4, S=0.001
    # Rectangular Channel: A=by, P=b+2y
    A = 4 * y
    P = 4 + 2 * y
    R = A / P
    Q_calc = (1/0.015) * A * (R**(2/3)) * (0.001**0.5)
    return Q_calc - 10  # Error (should be 0)

# Guess depth = 1.0m
depth = fsolve(manning_error, 1.0)
print(f"Required Depth: {depth[0]:.3f} m")
```

3.5 Chapter 9 Review Questions

1. Why do we check for $f(a) \times f(b) < 0$ in the Bisection method?
2. What is the main risk of using the Newton-Raphson method? (Hint: Look at the denominator of the formula).
3. Why are numerical methods necessary for hydraulic channel design?

Comprehensive Assignment 3 (Modules 7-9)

Instructions

This assignment integrates Data Science and Numerical Methods. Submit solution scripts for each question.

Question 1: Hydrological Data Analysis (Pandas)

You are provided with a CSV file `river_flow.csv` containing columns: `Date`, `Flow_m3s`, `Turbidity`. **Task:**

1. Load the data. Convert the Date column to datetime objects.
2. Identify missing values in the Flow column and fill them with the average flow of the dataset.
3. Filter the data to find all days where Flow > 100 m^3/s AND Turbidity > 50 NTU (Flood Events).
4. Create a new column `Status` that says "High" if Flow > 100, else "Normal".
5. Save this processed data to `processed_river.csv`.

Question 2: Climate Visualization Dashboard

Using `sua_weather_data.csv`: **Task:** Create a single figure with 2 subplots (stacked).

- **Subplot 1:** A Line plot of Max Temperature (Red) and Min Temperature (Blue) over time. Add a legend.
- **Subplot 2:** A Bar chart of Daily Rainfall (Green).
- Ensure all axes have labels (Date, Temp C, Rainfall mm) and the figure has a main title "SUA Weather Station Report". Save as `dashboard.png`.

Question 3: Friction Factor (Colebrook-White)

The friction factor f in a pipe is given by the implicit Colebrook-White equation:

$$\frac{1}{\sqrt{f}} = -2\log_{10}\left(\frac{\varepsilon}{3.7D} + \frac{2.51}{Re\sqrt{f}}\right)$$

This cannot be rearranged to isolate f. **Inputs:** Roughness $\varepsilon = 0.0002m$, Diameter $D = 0.5m$, Reynolds Number $Re = 100,000$. **Task:**

1. Define the error function $g(f) = LHS - RHS$.
2. Use `scipy.optimize.fsolve` to find f. (Initial guess: 0.02).

Question 4: Projectile Motion Root Finding

An irrigation sprinkler shoots water at $20m/s$ at an angle of 45°. The height y at distance x is:

$$y = x\tan(\theta) - \frac{gx^2}{2v^2\cos^2(\theta)}$$

Task: Find the distance x where the water hits the ground ($y = 0$).

1. Write the function for $y(x)$.
2. Use the **Bisection Method** (write your own function) to find the root between $x = 10$ and $x = 100$.
3. Tolerance should be 0.01 meters.

Question 5: Crop Yield Heatmap

You have a 5x5 grid representing yield (tons/ha) in different experimental plots. `yields = np.array([[3.2, 3.5, 3.1...], ...])` (Generate random data between 2.0 and 5.0). **Task:**

1. Use **Seaborn** to create a Heatmap of the yield.
2. Use a color map that makes high yield green and low yield red (`RdYlGn`).
3. Add annotations (the numbers) to each cell.
4. Save the image.